

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №34

Тема: Модульна контрольна робота (МКР) з тем 3–5: базові структури, обчислювальні та основні алгоритми

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Перевірити рівень засвоєння матеріалу тем 3–5 (практичні роботи №17–33): знання базових алгоритмічних структур (слідування, розгалуження, цикл), обчислювальних алгоритмів (алгоритм Евкліда, табулювання, суми, інтеграли, рівняння), основних алгоритмів (сортування, пошук, динамічне програмування, жадібні алгоритми, геометрія, опрацювання тексту), вміння трасувати алгоритми вручну та реалізовувати їх мовою Python.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

Модульна контрольна робота охоплює такий матеріал; для підготовки повторіть відповідні практичні роботи та контрольні питання до них:

Тема	Практичні роботи	Ключові поняття для повторення
3. Базові структури	№17–22	властивості алгоритму; блок-схеми; трасування; лінійні алгоритми, // і %; похибки float; розгалуження, логічні операції, граничні значення; цикли while/for, break/continue; повний перебір
4. Обчислювальні алгоритми	№23–26	алгоритм Евкліда (віднімання, залишки), розширений алгоритм; табулювання; суми й добутки, ряди з точністю ϵ ; методи прямокутників, трапецій, Сімпсона; бісекція, ітерації, метод Ньютона
5. Основні алгоритми	№27–33	сортування (вставки, вибір, бульбашка, Шелла, швидке, злиття, піраміда, підрахунком), стійкість; бінарний пошук, злиття; динамічне програмування (рюкзак, LCS, відстань Левенштейна); жадібні алгоритми; геометрія (cross, площа, оболонка); рядки, частоти, шифр Цезаря, RLE; вибір методу, стрес-тест

Організація роботи. Тривалість — 80 хвилин. Кожен студент виконує завдання **свого варіанта** (1–4, за порядковим номером у списку групи, номер варіанта = номер у списку mod 4, де 0 відповідає варіанту 4). Користуватись конспектом, довідниками та інтернетом заборонено; дозволено використовувати документацію мови Python (довідка `help()`) та калькулятор. Рекомендований розподіл часу: частина А — 10 хв, частина Б — 30 хв, частина В — 40 хв.

Структура роботи. Кожен варіант складається з трьох частин:

- **Частина А** — 5 тестових запитань із однією правильною відповіддю (по 4 бали, разом 20 балів);

- **Частина Б** — 2 завдання на ручне трасування алгоритмів із записом таблиці трасування (по 15 балів, разом 30 балів);
 - **Частина В** — програмне завдання: реалізація функцій мовою Python із тестами (50 балів).
-

Хід роботи

Варіант 1

Частина А. Тести (виберіть одну правильну відповідь)

1. Яка складність бінарного пошуку в упорядкованому масиві з n елементів? а) $O(n)$; б) $O(\log n)$; в) $O(n \log n)$; г) $O(1)$.
2. Скільки разів виконається тіло циклу `for i in range(3, 20, 4)`? а) 4; б) 5; в) 6; г) 17.
3. Чому дорівнює значення виразу `17 // 5 + 17 % 5 * 2`? а) 5; б) 7; в) 8; г) 11.
4. Який з наведених алгоритмів сортування є нестійким? а) вставками; б) злиттям; в) бульбашкове; г) пірамідальне.
5. Яка властивість задачі дозволяє застосувати динамічне програмування? а) наявність перекривних підзадач і оптимальної підструктури; б) задача має лише один параметр; в) вхідні дані впорядковані; г) відповідь — ціле число.

Частина Б. Ручне трасування

Б1. Виконайте вручну фрагмент та запишіть таблицю трасування ($k, k \% 3 == 0?, s$) і значення s та k після завершення циклу:

```
s = 0
k = 1
while k <= 20:
    if k % 3 == 0:
        s += k
    k += 2
```

Б2. Виконайте вручну **оптимізоване** бульбашкове сортування (з прапорцем `swapped`) масиву `[5, 2, 9, 1, 7]` за зростанням. Для кожного проходу запишіть стан масиву, кількість порівнянь і обмінів; вкажіть, на якому проході алгоритм завершиться, і загальну кількість порівнянь.

Частина В. Програмування

Напишіть функції `is_prime(n)` (перебір дільників до $\sqrt{n}$) і `count_primes(n)` (кількість простих чисел від 2 до n включно). Перевірте: `count_primes(30) → 10`, `count_primes(100) → 25`; `is_prime(1)`, `is_prime(2)`, `is_prime(97)`. Підрахуйте кількість операцій ділення при $n = 1000$ для варіанта з дільниками до $n-1$ і до $\sqrt{n}$ та порівняйте.

Варіант 2

Частина А. Тести (виберіть одну правильну відповідь)

1. Яка середня складність швидкого сортування (QuickSort)? а) $O(n)$; б) $O(n^2)$; в) $O(n \log n)$; г) $O(\log n)$.
2. Чому дорівнює значення виразу `-7 // 2` у Python? а) -3 ; б) -4 ; в) 3 ; г) -3.5 .
3. Що виведе `print(2 ** 3 ** 2)`? а) 64; б) 512; в) 36; г) 128.

4. Скільки операцій ділення з остачею виконає алгоритм Евкліда (варіант із залишками) для пари (48, 18) до отримання остачі 0? а) 2; б) 3; в) 4; г) 6.
5. Для якої задачі жадібний алгоритм у загальному випадку НЕ гарантує оптимуму? а) вибір максимальної кількості неперетинних активностей; б) неперервний рюкзак; в) рюкзак 0/1; г) кодування Гаффмана.

Частина Б. Ручне трасування

Б1. Знайдіть НСД(252, 105) алгоритмом Евкліда (варіант із залишками): складіть таблицю «крок — a — b — $a \bmod b$ », запишіть НСД та НСК(252, 105).

Б2. Виконайте вручну сортування вставками масиву [4, 3, 2, 10, 12, 1]: для кожного кроку запишіть вставлений елемент, стан масиву й кількість порівнянь на цьому кроці; підрахуйте загальну кількість порівнянь.

Частина В. Програмування

Напишіть функцію `min_coins(coins, amount)`, що динамічним програмуванням знаходить мінімальну кількість монет для суми та набір монет (або `None`, якщо неможливо), і функцію `greedy_change(coins, amount)`. Перевірте на системі (1, 3, 4) для суми 6 та на системі (2, 4) для суми 7. Знайдіть усі суми від 1 до 30, для яких жадібний розмін системою (1, 3, 4) не є оптимальним.

Варіант 3

Частина А. Тести (виберіть одну правильну відповідь)

1. Яка структура циклу перевіряє умову ПЕРЕД кожною ітерацією? а) цикл із постумовою; б) `while`; в) `try/except`; г) умовний вираз.
2. Чому дорівнює значення виразу `0.1 + 0.2 == 0.3` у Python? а) `True`; б) `False`; в) помилка виконання; г) `0.3`.
3. Яка складність сортування злиттям у найгіршому випадку? а) $O(n)$; б) $O(n^2)$; в) $O(n \log n)$; г) $O(\log n)$.
4. До чого призведе рекурсивна функція без базового випадку? а) поверне 0; б) `RecursionError` (переповнення стека викликів); в) виконається один раз; г) нічого, інтерпретатор виправить сам.
5. Яка формула задає метод Ньютона для рівняння $f(x) = 0$? а) $x_{k+1} = (a + b)/2$; б) $x_{k+1} = x_k - f(x_k)/f'(x_k)$; в) $x_{k+1} = f(x_k)$; г) $x_{k+1} = x_k + h$.

Частина Б. Ручне трасування

Б1. Виконайте вручну розбиття Ломута (півот – останній елемент) для масиву [7, 2, 9, 4, 6]: складіть таблицю « $j - a[j] - a[j] \leq \text{pivot?} - i$ – масив» і запишіть масив після розбиття та позицію півота.

Б2. Виконайте бінарний пошук числа 31 у масиві [3, 8, 15, 22, 31, 40, 57, 64]: складіть таблицю « $\text{left} - \text{right} - \text{mid} - a[\text{mid}]$ » і запишіть знайдений індекс та кількість порівнянь.

Частина В. Програмування

Напишіть функцію `max_subarray(a)` (алгоритм Кадане), що повертає найбільшу суму неперервного підмасиву та сам підмасив, і наївну функцію `max_subarray_naive(a)`. Перевірте на `[-2, 1, -3, 4, -1, 2, 1, -5, 4]` (сума 6, підмасив [4, -1, 2,

1]) та $[-8, -3, -6]$. Виконайте стрес-тест на 300 випадкових масивах і виведіть кількість розбіжностей.

Варіант 4

Частина А. Тести (виберіть одну правильну відповідь)

1. Яка складність обчислення відстані Левенштейна для рядків довжиною n і m динамічним програмуванням? а) $O(n + m)$; б) $O(n \cdot m)$; в) $O(2^n)$; г) $O(\log n)$.
2. Що буде результатом `"abc" * 2 + "d"`? а) `abcabcd`; б) `abc2d`; в) `abcabc d`; г) помилка.
3. Яка структура даних Python забезпечує пошук елемента в середньому за $O(1)$? а) список; б) кортеж; в) множина (`set`); г) рядок.
4. Чому дорівнює сума $1 + 2 + \dots + 100$? а) 5000; б) 5050; в) 5100; г) 10100.
5. Для чого використовується мемоізація? а) для зменшення розміру вхідних даних; б) для запам'ятовування результатів підзадач, щоб не обчислювати їх повторно; в) для сортування; г) для перевірки типів.

Частина Б. Ручне трасування

Б1. Виконайте вручну фрагмент і запишіть таблицю трасування $(i, j, (i + j) \% 2 == 0?)$, с) та кінцеве значення `c`:

```
c = 0
for i in range(1, 6):
    for j in range(i, 6):
        if (i + j) % 2 == 0:
            c += 1
```

Б2. Розв'яжіть неперервну задачу про рюкзак місткістю $W = 40$ для предметів (вага; цінність): (10; 30), (20; 100), (30; 120): обчисліть питому цінність, виберіть предмети жадібно (останній — частково) і знайдіть загальну цінність.

Частина В. Програмування

Напишіть функції `rle_encode(s)` і `rle_decode(s)` (стиснення повторів), а також `is_palindrome(text)` (з нормалізацією: лише літери та цифри, без урахування регістру). Перевірте: `rle_encode("aaabccddddd")` → `a3b1c2d4`; розпакування відновлює рядок; «А роза упала на лапу Азора» — паліндром. Знайдіть рядок, для якого RLE збільшує довжину.

Критерії оцінювання

Частина	Завдання	Максимум балів	Критерії
А	5 тестів	20	по 4 бали за правильну відповідь; неправильна або відсутня відповідь — 0 балів
Б	Б1, Б2	30 (по 15)	правильна таблиця трасування — 10; правильний остаточний результат — 3; пояснення / висновок — 2
В	Програма	50	коректність алгоритму — 25; відповідність результатів тестам — 10; обробка граничних випадків — 5; якість коду (назви, структура) — 5; висновок / пояснення — 5

Сума балів	Оцінка за 4-бальною шкалою	Рівень
90–100	5 (відмінно)	високий
74–89	4 (добре)	достатній
60–73	3 (задовільно)	середній
менше 60	2 (незадовільно)	низький — потрібне перескладання

Ключ до завдань (для викладача)

Частина А — правильні відповіді:

Варіант	1	2	3	4	5
1	б	б	б	г	а
2	в	б	б	б	в
3	б	б	в	б	б
4	б	а	в	б	б

Варіант 1. Б1: для $k = 1, 3, 5, \dots, 19$ умова $k \% 3 == 0$ істинна при $k = 3, 9, 15$, тому $s = 27$; після завершення $k = 21$. Б2: прохід 1: [2, 5, 1, 7, 9], порівнянь 4, обмінів 3; прохід 2: [2, 1, 5, 7, 9], порівнянь 3, обмінів 1; прохід 3: [1, 2, 5, 7, 9], порівнянь 2, обмінів 1; прохід 4: [1, 2, 5, 7, 9], порівнянь 1, обмінів 0. Алгоритм завершується на проході 4 (обмінів немає); загалом **10 порівнянь** і 5 обмінів. В: $\text{count_primes}(30) = 10$, $\text{count_primes}(100) = 25$.

Варіант 2. Б1: $(252, 105) \rightarrow 42$; $(105, 42) \rightarrow 21$; $(42, 21) \rightarrow 0$. НСД = 21, НСК = $252 \cdot 105 / 21 = 1260$. Б2: вставка 3: [3, 4, 2, 10, 12, 1], порівнянь 1; вставка 2: [2, 3, 4, 10, 12, 1], порівнянь 2; вставка 10: [2, 3, 4, 10, 12, 1], порівнянь 1; вставка 12: [2, 3, 4, 10, 12, 1], порівнянь 1; вставка 1: [1, 2, 3, 4, 10, 12], порівнянь 5. Загалом **10 порівнянь**. В: для (1, 3, 4) і суми 6 — 2 монети (3 + 3), жадібний розмін дає $4 + 1 + 1$; для (2, 4) і суми 7 — None.

Варіант 3. Б1: $j = 0$, $a[j] = 7$, ні, $i = -1$, [7, 2, 9, 4, 6]; $j = 1$, $a[j] = 2$, так, $i = 0$, [2, 7, 9, 4, 6]; $j = 2$, $a[j] = 9$, ні, $i = 0$, [2, 7, 9, 4, 6]; $j = 3$, $a[j] = 4$, так, $i = 1$, [2, 4, 9, 7, 6]; після обміну пивота: [2, 4, 6, 7, 9], позиція пивота 2. Б2: left = 0, right = 7, mid = 3, $a[\text{mid}] = 22$; left = 4, right = 7, mid = 5, $a[\text{mid}] = 40$; left = 4, right = 4, mid = 4, $a[\text{mid}] = 31$; знайдено індекс 4 за 3 порівняння(нь). В: Кадане – сума 6, підмасив [4, -1, 2, 1]; для [-8, -3, -6] – сума -3, підмасив [-3]; розбіжностей у стрес-тесті – 0.

Варіант 4. Б1: умова $(i + j) \% 2 == 0$ виконується для пар (1, 1), (1, 3), (1, 5), (2, 2), (2, 4), (3, 3), (3, 5), (4, 4), (5, 5); кінцеве значення $s = 9$. Б2: питоми цінності 3, 5, 4 (для 1, 2, 3 предметів); беремо предмет 2 повністю (вага 20, цінність 100), далі предмет 3 на 20 одиниць ваги (цінність $20 \cdot 4 = 80$), загальна цінність **180**. В: $\text{rle_encode}(\text{"aaabccddddd"}) = \text{a3b1c2d4}$; для рядка «abcd» RLE дає a1b1c1d1 — довша.

Еталонні реалізації програмних завдань (частина В) викладач може взяти з практичних робіт: №17–22 (цикли, перебір), №24–25, №27–28 (сортування, бінарний пошук), №29 (ДП, монети), №30 (жадібні алгоритми), №32 (рядки, RLE), №33 (Кадане, стрес-тест).

Контрольні питання

1. Назвіть основні властивості алгоритму та базові алгоритмічні структури.

2. Як побудувати таблицю трасування для циклу? Які значення в ній записують на кожному кроці?
3. Які особливості чисел із плаваючою комою треба враховувати при порівнянні та підсумовуванні?
4. Порівняйте алгоритм Евкліда з відніманням і з залишками. Яка складність другого?
5. Опишіть методи чисельного інтегрування та розв'язування рівнянь, які ви вивчали; порівняйте їх порядки збіжності.
6. Порівняйте алгоритми сортування за складністю, стійкістю та пам'яттю. Як обрати метод сортування?
7. У чому суть динамічного програмування? Чим воно відрізняється від жадібного підходу?
8. Як за постановкою нової задачі обрати метод її розв'язання та перевірити результат?

Зміст звіту

1. Тема, мета роботи, номер варіанта.
2. Відповіді на тестові запитання (частина А).
3. Таблиці трасування та остаточні результати (частина Б).
4. Тексти програм та результати їх роботи (частина В).
5. Висновки.